Contents

Introduction

This document contains commonly used code for competitive programming, specifically in rust and written for the 2026 finale in the norwegian olympiad of informatics.

Basics

Rust boilerplate

```rust
use std::convert::TryInto;
use std::io;

fn main() {
  let mut lines = io::stdin().lines();
}
```

The program can then be compiled and run with

```
rustc -O -o main main.rs && ./main < 1.txt
```

Capturing input

We will use the stdin lines iterator defined above to read the input:

```
let n: usize = lines.next().unwrap().unwrap().parse();
```

For reading multiple variables in a single line, one can do the following:

```
let [n, q]: [usize; 2] = lines
    .next()
    .unwrap()
    .unwrap()
    .trim()
    .split_whitespace()
    .map(|x| x.parse().unwrap())
    .collect::<Vec<_>>()
    .try_into()
    .unwrap();
```

, which is expandable to any number of values in the same line.

Note

This is unoptimal as it allocates a `Vec` when it is not necessary, but in my experience this was negligible. It can however be improved as such:

```
let mut line = lines.next().unwrap().unwrap().split_whitespace();
let n: usize = iter.next().unwrap().parse().unwrap();
let q: usize = iter.next().unwrap().parse().unwrap();
```

Bitmasks and bitwise operations (TODO)

Operator	Operation
`&`	AND
`|`	OR
`^`	XOR
`!`	NOT
`<<`	Left shift
`>>`	Right shift

Match

Useful for pattern matching. Works similar to that of languages like haskell or python. It can contain expressions and also return values directly.

```
let bar = match foo {
    Some(x) = x,
```

```
    None = 0,
}
```

```
match foo {
    Some(0) = {
        println!("Value is zero :o");
    },
    Some(x) if x < 10 {
        println!("X is less than 10");
    },
    Some(x) {
        println!("Value: {}", x);
    },
    None = {
        println!("No value :(");
    }
}
```

Note

One can use the operator `@` to set a value while matching a pattern. For example, one can combine the pattern `x` and `1..=10` into a single expression with `x @ 1..=10`, which will keep the original value in `x` while matching it against the pattern.

Time complexity

Depending on the constraints in the problem, different algorithms should be chosen, approximately based on this:

n	Time complexity
1 000 000	$O(n)$
400 000	$O(n \log n)$
1 000	$O(n^2)$
200	$O(n^3)$
20	$O(2^n)$
10	$O(n!)$

Useful data types

Option

An option holds an optional value, similar to the `Maybe`-monad in haskell:

```
let x: Option<i32> = None;
let x: Option<i32> = Some(42);
```

```
match x {
  Some(n) => println!("value exists"),
  None => println!("no value"),
}

let x: i32 = x.unwrap(); // unsafe; panics if None
let x: i32 = x.unwrap_or(0); // safe; adds a fallback value
let x: Option<i32> = x.map(|v| v * 2);
```

Enums

Can be defined with

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum Instruction {
    Build,
    Query,
}
```

Vectors

Vectors are lists of items with a dynamic length. Here are some examples:

```
let xs: Vec<i32> = vec![1, 2, 3, 4];
let xs: Vec<i32> = vec![0; 5]; // [0, 0, 0, 0, 0]
let mut xs: Vec<usize> = Vec::new(); // xs = vec![]
xs.push(42); // xs = vec![42]
xs.push(123); // xs = vec![42, 123]
```

VecDeque

For algorithms such as 0-1 BFS, one can use a double-ended queue as follows:

```
use std::collections::VecDeque;

let mut queue: VecDeque<usize> = VecDeque::new();

while let Some(x) = queue.pop_front() {
  if weights[x] == 0 {
    for &n in &graph[x] {
      queue.push_front(n);
    }
  } else {
    for &n in &neighbors[x] {
      queue.push_back(n);
    }
  }
}
```

Note that this is just pseudocode.

Slices

A vector can be indexed with a slice as such:

```
let xs = vec![0, 1, 2, 3, 4];

let slice = &xs[1..4]; // [1, 2, 3]
let slice = &xs[1..=3]; // [1, 2, 3]
let slice = &xs[..3]; // [0, 1, 2]
let slice = &xs[2..]; // [2, 3, 4]
let slice = &xs[..]; // [0, 1, 2, 3, 4]
```

Slices can also be mutable:

```
let xs = vec![0, 1, 2, 3, 4];

for x in &mut xs[1..4] {
  *x *= 2;
}

println!("{:?}", xs); // [0, 2, 4, 6, 4]
```

Note

Slices borrow from the vector. `xs[1..4]` produces a slice of type `&[i32]`.

A slice can also be used as an iterator, in pattern matching, etc.

```
let xs = vec![0, 1, 2, 3, 4];

match &xs[..] {
    [first, .., last] => {
        println!("First: {}, Last: {}", first, last);
    }
}

// .iter() on a slice of type &[i32] creates an iterator of references (`&i32`)
let slice_sum = xs[1..4].iter().sum(); // 6
```

Sets and maps

HashSet

Note that most set operations use borrows, with the exception of `insert`.

```
use std::collections::HashSet;

let mut set: HashSet<usize> = HashSet::new();

set.insert(5);
```

```
set.insert(6);

assert!(set.contains(&5));
assert!(!set.contains(&7));

set.remove(&6);
```

Note

When doing a lookup, one uses a reference to the value, not the value itself (`set.contains(&5)`). The same applies to HashMaps.

HashMap

HashMaps store data in a map with hashed keys and have $O(1)$ operations.

```
use std::collections::HashMap;

let mut map: HashMap<usize, &str> = HashMap::new();
map.insert(1, "one");
map.insert(2, "two");

let x: Option<&str> = map.get(&1); // Some("one")
let x: Option<&str> = map.get(&3); // None

assert!(map.contains_key(&2));
map.remove(&2);
assert!(!map.contains_key(&2));

map.insert(1, "en");
```

There is also more complex funcionality present, such as:

```
let mut map: HashMap<usize, i32> = HashMap::new();
map.insert(1, 123);

let x: &i32 = map[&1]; // unsafe; panics if 1 does not exist
let x: Option<&i32> = map.get(&1); // safe

let mut x: &mut i32 = map.entry(1); // .entry() returns an &mut i32
let x: &i32 = map.entry(1).or_insert(456); // 123
let x: &i32 = map.entry(2).or_insert(456); // 456

*map.entry(3).or_insert(0) += 1; // map[&3] == 1
*map.entry(3).or_insert(0) += 1; // map[&3] == 2

map.entry(4).and_modify(|ref mut v| v += 1).or_insert(1); // map[&4] = 1
map.entry(4).and_modify(|v| *v += 1).or_insert(1); // map[&4] = 2
```

BTreeSet and BTreeMap

BTreeSet and BTreeMap work conceptually same as their Hash counterparts, with the addition that they are automatically sorted. The difference is that the Hash versions are (as the name suggests) hash-based with $O(1)$ operations, and the BTree versions have $O(\log n)$ operations..

Note

These data types internally use a B-tree to sort the values, hence the name “B-Tree Set”.

Here are some usage examples:

```
use std::collections::BTreeSet;

let mut set = BTreeSet::new();
set.insert(10);
set.insert(5);
set.insert(20);

let left: Option<&i32> = set.range(..10).next_back();
let right: Option<&i32> = set.range(10 + 1..).next();
```

If one needs to store data along with the sorted indexes, a BTreeMap is useful. It would also work to use a separate HashMap and BTreeSet, but combining it into a BTreeMap is more efficient and makes the code cleaner.

```
use std::collections::BTreeMap;

let mut map = BTreeMap::new();
map.insert(10, "root");
map.insert(5, "left child");
map.insert(20, "right child");

let left_depth: Option<&&str> = map.range(..10).next_back().map(|(_, v)| v);
// Optionally:
let left_depth: Option<&str> = map.range(..10).next_back().map(|(_, v)| *v);
```

Data structures

Graphs

I like to define graphs as follows:

```
type Graph = Vec<Vec<(usize, usize)>>;
```

Which will be indexed like this:

```
graph[from_node] = [(neighbor1, cost), (neigbor2, cost), (neighbor3, cost)]
```

Note

Unweighted graphs can be simplified to

```
type Graph = Vec<Vec<usize>>;
```

Here is an example of how a graph can be constructed in rust (taken from my solution to `nio21-finale-togtur`). In this case I am creating an undirected graph. For a directed graph, one would remove the line with `graph[b].push((a, k))`.

```
let mut graph: Vec<Vec<(usize, usize)>> = vec![Vec::new(); n];
for _ in 0..m {
    let [a, b, k]: [usize; 3] = lines
        .next()
        .unwrap()
        .unwrap()
        .trim()
        .split_whitespace()
        .map(|x| x.parse().unwrap())
        .collect::<Vec<_>>()
        .try_into()
        .unwrap();
    graph[a].push((b, k));
    graph[b].push((a, k));
}
```

Dijkstra

Dijkstra's algorithm is an algorithm for finding the lowest total weight path between two nodes in a graph. It has a time complexity of $O((V+E)\log V)$. It requires the following imports:

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;
```

Note

We use `Reverse` here because rust's `BinaryHeap` sorts the values descending, but for Dijkstra we want it to be ascending. `Reverse` is used to reverse the natural ordering of values.

And can be implemented as such:

```
let n: usize; // Number of nodes in the graph
let graph: Graph;
let start: usize;
let target: usize;

let mut heap = BinaryHeap::new();
heap.push(Reverse((0, start)));
let mut dist = vec![usize::MAX; n];
```

```
dist[start] = 0;

let mut result = None;

while let Some(Reverse((cost, node))) = heap.pop() {
    if cost > dist[node] {
        continue;
    }

    if node == target {
        result = Some(cost);
        break;
    }

    for &(neighbor, c) in &graph[node] {
        let new_cost = cost + c;
        if new_cost < dist[neighbor] {
            dist[neighbor] = new_cost;
            heap.push(Reverse((new_cost, neighbor)));
        }
    }
}

println!("{}", result.unwrap());
```

There are often problems which require a variation of Dijkstra, for example the aforementioned `nio21-finale-togtur` which adds another layer for each “free edge”. This can easily be implemented by extending the dist DP table and adding another item to the tuple in the heap:

```
heap.push(Reverse((0, v, start))); // (cost, tickets, node)
let mut dist = vec![vec![usize::MAX; v + 1]; n];
dist[start][v] = 0;

// ---

for &(n, c) in &graph[node] {
    let new_cost = cost + c;
    if new_cost < dist[n][tickets] {
        dist[n][tickets] = new_cost;
        heap.push(Reverse((new_cost, tickets, n)));
    }

    if tickets > 0 && cost < dist[n][tickets - 1] {
        dist[n][tickets - 1] = cost;
        heap.push(Reverse((cost, tickets - 1, n)));
    }
}
```

BFS (TODO)

DFS (TODO)

DP (TODO)

Binary trees

A binary tree can be defined with

```
#[derive(Debug)]
struct Node<T> {
    value: T,
    left: Option<Box<Node<T>>>,
    right: Option<Box<Node<T>>>,
}

impl<T> Node<T> {
    fn new(value: T) -> Self {
        Node {
            value,
            left: None,
            right: None,
        }
    }
}
```

Which can then be used like

```
let mut root = Node::new(10);

root.left = Some(Box::new(Node::new(5)));
root.right = Some(Box::new(Node::new(20)));

dbg!(root);
```

Traversing it can be done either recursively or iteratively, as follows:

```
fn inorder<T: std::fmt::Display>(node: &Option<Box<Node<T>>>) {
    if let Some(n) = node {
        inorder(&n.left);
        print!("{} ", n.value);
        inorder(&n.right);
    }
}
```

Here is an example taken from my unoptimized solution to `nio24-finale-trebygger`:

```
let mut root: Node<usize> = Node::new(xs[0]);
println!("0");
```

```rust
for &a in &xs[1..] {
    let mut h = 1;
    let mut c = &mut root;

    loop {
        if a < c.value {
            match c.left {
                Some(ref mut l) => {
                    c = l;
                }
                None => {
                    c.left = Some(Box::new(Node::new(a)));
                    println!("{}", h);
                    break;
                }
            }
        } else {
            match c.right {
                Some(ref mut r) => {
                    c = r;
                }
                None => {
                    c.right = Some(Box::new(Node::new(a)));
                    println!("{}", h);
                    break;
                }
            }
        }

        h += 1;
    }
}
```

Note

One usually will not need to implement a tree in CP - most of the time it will suffice to use a `BTreeSet` or `BTreeMap`. The solution listed above exists as an example of how one *could* use this data structure, but in this and most other cases it will be better to use one of the aforementioned data types. The above solution could be significantly optimized to this:

```rust
let mut tree: BTreeMap<usize, usize> = BTreeMap::new();

tree.insert(xs[0], 0);
println!("0");

for &x in &xs[1..] {
    let l = tree.range(..x).next_back().map(|(_, &d)| d);
    let r = tree.range(x + 1..).next().map(|(_, &d)| d);

    let d = l.unwrap_or(0).max(r.unwrap_or(0)) + 1;
```

```
    tree.insert(x, d);

    println!("{}", d);
}
```

Difference arrays

Difference arrays provide a more efficient way to update all values within an interval. Instead of looping over all items in the interval $[a, b]$, one instead just sets the values at a and b, then passes over the array afterwards. A boolean implementation could look like this (taken from my solution to `nio23-runde2-lynnedslag`):

```
let mut houses: Vec<bool> = vec![false; n];

for _ in k {
  let [a, b]: [usize; 2];
  houses[a] = !houses[a];
  houses[b+1] = !houses[b+1];
}

let mut flip = false;
let mut result = 0;

for h in houses {
  flip ^= h;

  if !flip {
    result += 1;
  }
}
```

Note

We use $b + 1$ instead of b because the interval is inclusive. The difference array marks where the effect starts and stops, and the prefix accumulation applies the effect from a through b.

A more general implementation could look like this:

```
let mut xs: Vec<isize> = vec![0; 10 + 1];

// Add 3 to the interval [2, 5]
xs[2] += 3;
xs[5+1] -= 3;

// Subtract 9 in the interval [6, 9]
xs[6] -= 9;
xs[9+1] += 9;
```

```
let mut d = 0;
let mut result = 0;

for x in xs {
  d += x;
  result += d;
}
```

Note that the previous version is actually a special case of this, in which one would use `+= x % 2` (which I have simplified to using booleans).

Algorithms

Binary search (TODO)